WEEK 4 · LESSON 22 OF 24 · HANDOUT

Planning Your Capstone Project

Study time: **about 60 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=4mGvH3ms6B8 — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyData Berlin 2023: How to baseline in NLP and where to go from there — https://www.youtube.com/watch?v=oc_p0ZgoyA8

🎓 **Go deeper:** Stanford Machine Learning, Lecture 1 — Andrew Ng — https://www.youtube.com/watch?v=UzxYlbK2c7E

Your goal in this lesson: Choose a valuable, feasible and responsible capstone problem, define success measures and baselines, plan data, risks and timeline, and get your plan approved.

What the capstone is

Your capstone is one end-to-end AI project that brings the whole WAMUGII AI Program together: a real problem, real or realistic data, a model (classic ML, deep learning or an LLM application), an API, tests, monitoring, a fairness and privacy review, documentation and a presentation.

Choosing the problem

A good capstone problem is:

1. **Valuable** — someone real has this problem and would use the result.
2. **Feasible** — you can get the data and build it in about two weeks.
3. **Measurable** — you can tell whether it works.
4. **Responsible** — you can manage its risks; avoid unacceptable uses.

Ideas that fit Rwanda and the region:

Idea	Type	Data
Crop-disease detection from leaf photos	Image classification (Course 5)	Public plant-disease image sets + your own photos
Support-ticket router for a company or school	Text classification or LLM (Courses 4–6)	Anonymised past tickets
Ask-the-handbook assistant for a school or SACCO	RAG (Course 6)	Policies you are allowed to use
Mobile-money transaction anomaly alerts	Anomaly detection (Course 4)	Synthetic or anonymised data
Student drop-out early warning	Classification + fairness (this course)	Anonymised school records, with permission
Clinic appointment no-show prediction	Classification	Anonymised records, with permission

The one-page canvas

```
CANVAS = {
    "Problem": "Small clinics lose time when patients miss appointments.",
    "Users": "Clinic receptionists, who call patients the day before.",
    "Decision supported": "Which patients to call first (never: refuse care).",
    "Success measure": "Recall of no-shows in the top 20% called, compared with calling at
random.",
    "Baseline to beat": "Call the patients with the most past no-shows.",
    "Data": "Anonymised appointment records from one partner clinic, with written permission.",
    "Model": "Logistic regression first; gradient boosting if clearly better.",
    "Delivery": "FastAPI service + simple daily call list.",
    "Risks": "Privacy of health data; unfair targeting by area; staff over-trust.",
    "Ethics checks": "Fairness by area and age; data protection review; human always decides.",
}
missing = [k for k, v in CANVAS.items() if not v.strip()]
print("canvas complete" if not missing else f"missing: {missing}")
for k, v in CANVAS.items():
    print(f"{k:20s} {v}")
```

Always beat a baseline

Before any model, compute what a simple rule achieves. If your model cannot beat it, it is not worth deploying:

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
tr, te = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants["repaid"])

majority = DummyClassifier(strategy="most_frequent").fit(tr[FEATURES], tr["repaid"])
rule = ((te["loan_rwf"] <= 2 * te["income_rwf"]) & (te["late_payments"] <= 1)).astype(int)
lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(tr[FEATURES],
tr["repaid"])

print(f"always 'repays':        accuracy {accuracy_score(te['repaid'],
majority.predict(te[FEATURES])):.3f}")
print(f"simple written rule:    accuracy {accuracy_score(te['repaid'], rule):.3f}")
print(f"logistic regression:    accuracy {accuracy_score(te['repaid'],
lr.predict(te[FEATURES])):.3f} "
      f"(AUC {roc_auc_score(te['repaid'], lr.predict_proba(te[FEATURES])[:, 1]):.3f})")
```

Guessing "everyone repays" is already right two times out of three, so "67% accuracy" would be worthless. Our model clearly beats both baselines — that is the evidence a project needs.

Plan the two weeks

```
plan = pd.DataFrame([
    ("Day 1-2", "Canvas approved, data access and permissions, datasheet draft"),
    ("Day 3-4", "Data cleaning and validation, baselines"),
    ("Day 5-7", "Models, experiment tracking, choose the simplest good model"),
    ("Day 8", "Fairness, explainability and privacy review"),
    ("Day 9-10", "API, tests, Docker package, monitoring plan"),
    ("Day 11", "Model card, risk register, impact assessment"),
    ("Day 12-13", "Report and presentation; rehearse the demo"),
    ("Day 14", "Present; submit code, report and documents"),
], columns=["When", "What"])
print(plan.to_string(index=False))
```

Get approval before you build

Send your instructor the canvas, data source and permission, the baseline, and your top three risks with controls. If a project uses personal data, health data or affects important decisions, extra care is needed — or choose synthetic data.

Key terms

- **Capstone** — a final project that combines everything you learned.
- **Project canvas** — a one-page plan of problem, users, data, measures and risks.
- **Baseline** — the result of a simple method your model must beat.
- **Success measure** — the number that shows whether the project works.
- **Feasibility** — whether the project can be done with the time and data available.

Remember this: Choose a valuable, feasible, measurable and responsible problem, define success and a baseline first, plan the two weeks, and get approval before building.

Name: _____ Date: _____

Activity

Task 1

Fill in the canvas for your own capstone idea.

Task 2

Write your success measure and baseline in one sentence each.

Task 3

List your data source, who owns it, and what permission you have.

Task 4

Post your canvas in the lesson Q&A and ask a classmate for one improvement.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Which is NOT one of the four qualities of a good capstone problem?
   - A. Feasible
   - B. Measurable
   - C. As complex as possible
   - D. Valuable
2. Why compute a baseline first?
   - A. It is required by Docker
   - B. To slow the project
   - C. Baselines are always best
   - D. Your model is only worth deploying if it beats a simple method
3. Guessing 'everyone repays' gave about what accuracy?
   - A. About 67%
   - B. 10%
   - C. 50% exactly
   - D. 100%
4. What should you send your instructor before building?
   - A. Your password
   - B. Only the final code
   - C. Canvas, data permission, baseline and top risks
   - D. Nothing
5. What should you do if your project needs sensitive personal data you cannot protect?
   - A. Use it anyway
   - B. Ignore privacy
   - C. Take extra care and get permission — or use synthetic data
   - D. Post it online
6. Which model should you try first in most capstones?
   - A. The biggest neural network
   - B. A random guess
   - C. No model
   - D. A simple model such as logistic regression

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.